TECNM; Instituto Tecnológico de Orizaba.

Materia: Programación Web

Grupo: 6g5C

Docente: María Angélica Martínez Herrera

Unidad: 1. Introducción a las aplicaciones web

Tarea: 3. Tecnologías para el desarrollo de aplicaciones web

Presenta equipo 1:

Arellano Crivelli Rafael

De Jesús Palacios Irais

Jürgen Jared Jiménez Vázquez

Vásquez Leal Irving

1. **Lenguajes de programación Web y DBMS.**

Existen numerosos lenguajes de programación empleados para el desarrollo de aplicaciones Web en el servidor, entre los que destacan:

- PHP
- Java, con sus tecnologías Java Servlets y JavaServer Pages (JSP)
- Java Script
- Perl
- Ruby
- Python
- C# y Visual Basic con sus tecnologías ASP/ASP.NET

También son muy utilizados otros lenguajes o arquitecturas que no son propiamente lenguajes de programación, como HTML o XML.

DBMS (Data Base Managment System)

Un **SMBD** (Sistema Manejador de Bases de Datos), también llamado **Gestor de Base de Datos**, es el software central que administra la creación, el acceso y el control total de la información. Su meta principal es permitir que el almacenamiento y la recuperación de datos sean prácticos y eficientes.

Componentes y Lenguajes

Para operar, un SMBD utiliza tres lenguajes clave:

- **DDL (Data Definition Language):** Para definir la estructura de los datos.
- **DML (Data Manipulation Language):** Para manipular los registros.
- **SQL (Structured Query Language):** El lenguaje estándar de consultas.

Funciones Principales

- **Organización:** Crea y mantiene las rutas de acceso rápido a los datos.
- **Gestión de Archivos:** Interactúa con el sistema de archivos para el almacenamiento físico real.
- **Control de Usuarios:** Atiende peticiones y registra el uso de la base de datos.
- **Seguridad Física:** Gestiona respaldos y recuperación ante fallas del sistema.

Objetivos del SGBD

Para que un gestor sea efectivo, debe garantizar:

- **Abstracción:** Oculta la complejidad física del almacenamiento al usuario.

- **Independencia:** Permite modificar la estructura de los datos sin afectar a las aplicaciones.
- **Consistencia:** Evita errores en datos repetidos y asegura que se cumplan las reglas del negocio (ej. validación de edades).
- **Seguridad:** Controla quién puede ver o editar la información mediante permisos.
- **Transacciones:** Asegura que las operaciones se completen totalmente o no se apliquen en absoluto (evitando datos a medias tras una falla).
- **Eficiencia:** Optimiza el tiempo de respuesta en cada consulta.

2. **Plataforma Web Multiplataforma**

Una plataforma web es un sistema que funciona desde un **navegador**, lo que permite su ejecución sin importar el sistema operativo (SO) del usuario.

- **Ventaja:** Elimina la necesidad de desarrollar software específico para cada sistema operativo.
- **Desventaja:** Tiene limitaciones de potencia al no disponer del acceso directo a los recursos del sistema operativo que posee una aplicación nativa.

3. **Servidor Web**

Definición de Servidor

Los **servidores** son dispositivos de red que brindan un servicio a otros dispositivos llamados **clientes**. Aunque técnicamente es un **software especializado**, comúnmente se denomina así al equipo físico donde se ejecuta. Cualquier computadora puede ser un servidor, pero existen **sistemas operativos especializados** (Windows Server, Debian, Solaris) y hardware diseñado para alto rendimiento las 24 horas.

Clasificación de Servidores

- **De archivos:** Centralizan la información para que los usuarios accedan a ella remotamente.
- **De aplicaciones:** Permiten el procesamiento de datos de una aplicación de cliente.
- **De correo:** Mueven y almacenan el correo electrónico a través de redes locales, extensas e Internet.

- **De base de datos:** Funcionan como interfaz entre la base de datos, el usuario y las aplicaciones.
- **Servidores Web:** Alojan sitios y aplicaciones a los que se accede mediante un navegador usando el protocolo **HTTP**.

Programación del lado del servidor

Un servidor web consta de un intérprete HTTP que responde a las peticiones del cliente. Además, puede disponer de intérpretes para lenguajes como **ASP, PHP, Perl y Ajax**. Esto permite la **programación de lado del servidor**, la cual ejecuta tareas complejas abstrayendo al cliente de toda la operación.

4. Seguridad

El Problema de la Seguridad Web

El crecimiento exponencial de Internet ha convertido a los **servidores Web** en el punto más crítico de seguridad debido a la sensibilidad de la información que manejan. Aunque existen fallas en software (Apache, IIS) o lenguajes, la mayoría de los problemas son provocados por **malos usos de los programadores** y la **escritura defectuosa de código** a nivel aplicación.

Fallas en el Desarrollo

La falta de seguridad suele derivarse de no prestar atención a dos aspectos fundamentales:

- **Entradas al sistema:** Ninguna entrada debe ser digna de confianza plena; todas deben pasar por un **filtrado y proceso de limpieza** para confirmar su usabilidad.
- **Salidas del sistema:** Es vital **escapar las salidas** (como la codificación en entidades HTML) para evitar que la información sea interpretada erróneamente en su nuevo contexto (navegador o base de datos).

Riesgos de Configuración e Información

- **register_globals (PHP):** Este parámetro es un riesgo porque **oculta el origen de los datos**, impidiendo saber si una variable entró por GET o POST y provocando la pérdida de control sobre los procesos de seguridad.
- **Fuga de información:** Los **mensajes de error** del servidor, útiles en desarrollo, deben deshabilitarse en producción, ya que los atacantes los utilizan para obtener datos del sistema.

Responsabilidad del Programador

La seguridad involucra principalmente al desarrollador. Este debe comprender que las aplicaciones pueden ser accedidas con herramientas distintas al navegador común y es su responsabilidad **rechazar o filtrar peticiones** que no cumplan con las características esperadas, manteniendo un control estricto sobre la codificación de los datos.

5. **Frameworks**

Es una estructura de software diseñada para apoyar el desarrollo de sitios web, incluyendo aplicaciones web, servicios web y recursos web. Su objetivo es automatizar las tareas usualmente asociadas con el desarrollo web (como el acceso a bases de datos, el manejo de plantillas y la gestión de sesiones), permitiendo que el desarrollador se centre en la lógica de su aplicación.

Clasificaciones:

Frontend (Lado del Cliente): Gestionan la interfaz de usuario (UI), el diseño y la interactividad en el navegador.

- **Ejemplos:** React, Angular, Vue.js, Svelte.

Backend (Lado del Servidor): Gestionan la lógica de negocio, la seguridad, las bases de datos y el enrutamiento.

- **Ejemplos:** Django (Python), Laravel (PHP), Express.js (Node.js), Spring Boot (Java).

Fullstack / Híbridos: Combinan capacidades de ambos lados, optimizando el rendimiento y el SEO.

- **Ejemplos:** Next.js, Remix, Astro.

6. **Interfaz de programación de aplicaciones (API)**

Las API son mecanismos que permiten a dos componentes de software comunicarse entre sí mediante un conjunto de definiciones y protocolos. Por ejemplo, el sistema de software del instituto de meteorología contiene datos meteorológicos diarios. La aplicación meteorológica de su teléfono “habla” con este sistema a través de las API y le muestra las actualizaciones meteorológicas diarias en su teléfono.

¿Cómo funcionan las API?

La arquitectura de las API suele explicarse en términos de cliente y servidor. La aplicación que envía la solicitud se llama cliente, y la que envía la respuesta se llama servidor. En el ejemplo del tiempo, la base de datos meteorológicos del instituto es el servidor y la aplicación móvil es el cliente.

Las API pueden funcionar de cuatro maneras diferentes, según el momento y el motivo de su creación;

- **API de SOAP:** Estas API utilizan el protocolo simple de acceso a objetos. El cliente y el servidor intercambian mensajes mediante XML. Se trata de una API menos flexible que era más popular en el pasado. **Ejemplo:**
    - **Servicios de Validación de Impuestos (SAT en México):** Muchos sistemas gubernamentales usan SOAP para que las empresas envíen facturas electrónicas de forma oficial.
- **API de RPC:** Estas API se denominan llamadas a procedimientos remotos. El cliente completa una función (o procedimiento) en el servidor, y el servidor devuelve el resultado al cliente. **Ejemplo:**
    - **Google Cloud Pub/Sub:** Utiliza **gRPC** (una versión moderna de RPC) para mover cantidades masivas de datos entre servicios de forma casi instantánea.
- **API de WebSocket:** La API de WebSocket es otro desarrollo moderno de la API web que utiliza objetos JSON para transmitir datos. La API de WebSocket admite la comunicación bidireccional entre las aplicaciones cliente y el servidor. El servidor puede enviar mensajes de devolución de llamada a los clientes conectados, por lo que es más eficiente que la API de REST. Ejemplos**:**
    - **Notificaciones de Facebook:** Cuando alguien te da "Like" y te aparece el globo rojo sin que recargues la página, es un WebSocket enviando la señal.
    - **Documentos colaborativos (Google Docs):** Permite que veas en tiempo real lo que otra persona está escribiendo en el mismo documento.
- **API de REST:** Estas son las API más populares y flexibles que se encuentran en la web actualmente. El cliente envía las solicitudes al servidor como datos. El servidor utiliza esta entrada del cliente para iniciar funciones internas y devuelve los datos de salida al cliente. Veamos las API de REST con más detalle a continuación. **Ejemplos:**
    - **API de PayPal:** Cuando pagas en una tienda online, la tienda se comunica con PayPal vía REST para confirmar que el pago fue exitoso.

Conclusión grupal:

El desarrollo de aplicaciones web modernas se fundamenta en la integración cohesiva de múltiples tecnologías que abarcan desde lenguajes de programación robustos y sistemas gestores de bases de datos (DBMS), hasta infraestructuras de servidor especializadas. Se observa que la eficiencia de estas aplicaciones no solo radica en la elección de un lenguaje en el servidor, como PHP, Python o Java, sino en la capacidad del desarrollador para implementar arquitecturas de comunicación precisas, tales como las APIs de tipo REST o WebSockets, que garanticen la interoperabilidad y el flujo de datos en tiempo real. Asimismo, la adopción de *frameworks* tanto en el *frontend* como en el *backend* se consolida como una práctica indispensable para estandarizar procesos, optimizar recursos y permitir que los esfuerzos se centren en la lógica de negocio.

No obstante, la implementación de estas herramientas conlleva una responsabilidad crítica en materia de seguridad. La conclusión fundamental es que la integridad de un sistema web depende directamente de la gestión de entradas y salidas de datos, así como de la correcta configuración del entorno de producción. Por tanto, el éxito en la creación de plataformas multiplataforma no solo exige el dominio técnico de las herramientas de desarrollo y la gestión eficiente de transacciones en bases de datos, sino también una postura proactiva y ética por parte del programador para mitigar vulnerabilidades y garantizar una experiencia de usuario segura, consistente y de alto rendimiento.

Referencias adicionales:

MDN. (s.f). Frameworks web del lado del servidor. MDN. Recuperado de:

https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/First_steps/Web_frameworks

Amazon Web Services. (s.f.). ¿Qué es una interfaz de programación de aplicaciones (API)? Amazon Web Services, Inc. Recuperado de: https://aws.amazon.com/es/what-is/api/